

Knowledge Graph Engineering for Multi-Agentic Systems: The Anthropic Playbook

A Synthesis for Study

Based on Anthropic's Knowledge Graph Cookbook, Building Effective AI Agents, and Claude API documentation

Independently compiled, July 2026 — not affiliated with or endorsed by Anthropic

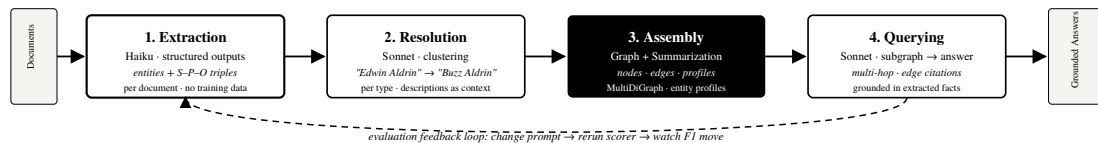


Fig. 1. The Knowledge Graph Pipeline. Four stages — extraction, resolution, assembly, querying — each implemented as Claude API calls. Haiku handles high-volume extraction; Sonnet handles resolution and reasoning. The evaluation feedback loop closes the circle.

Abstract—Multi-agent systems can generate, evaluate, and orchestrate work at machine speed, but they share a fundamental weakness: each agent's memory dies with its context window. When agents need to reason across documents, chain facts that never co-occur in a single source, or maintain a shared world model across sessions, the context window is not enough. This note presents **KNOWLEDGE GRAPH ENGINEERING** as the missing infrastructure layer for multi-agentic systems. We show how Claude replaces an entire classical NLP pipeline — trained NER, trained relation classifier, hand-written entity-resolution heuristics — with a sequence of structured-output prompts that extract typed entities and subject-predicate-object triples, resolve surface-form variants into canonical nodes, assemble a queryable graph, and answer multi-hop questions with edge-level citations. We ground the construction in Anthropic's published agent patterns (augmented LLM, orchestrator-workers, evaluator-optimizer) and show where the knowledge graph slots into each: as the shared memory for orchestrator-workers, as the grounding layer for evaluator-optimizer loops, and as the persistent world model that lets a loop pick up today where it left off yesterday. We report extraction quality (precision/recall against a gold set), discuss the cost/quality tradeoff between Haiku and Sonnet, and give scaling guidance for production graphs.

Index Terms—Knowledge graphs, named entity recognition, entity resolution, multi-agent systems, structured outputs, Claude, agentic AI, graph-grounded reasoning.

I. INTRODUCTION

You have a pile of unstructured documents and need to answer questions that span them — "who works with people who worked on project X," "which vendors are connected to this incident." No single document contains the answer. Retrieval-augmented generation can surface relevant chunks, but it cannot chain the facts for you. What you need is a knowledge graph: **entities as nodes, typed relations as edges**, so that multi-hop reasoning becomes graph traversal.

Building one used to mean training a named-entity recognizer on your domain, training a relation classifier, writing entity-

resolution heuristics, and maintaining all three as your data shifted. With Claude, each of those stages becomes a prompt. The entire classical NLP pipeline collapses into a sequence of structured-output calls whose "training data" is a Pydantic schema.

This matters especially for **multi-agent systems**. Anthropic's own engineering guidance describes agents as "typically just LLMs using tools based on environmental feedback in a loop," and their most-cited advice is to prefer "simple, composable patterns rather than complex frameworks." The five canonical patterns — the **augmented LLM**, **prompt chaining**, **routing**, **orchestrator-workers**, and **evaluator-optimizer** — are the vocabulary for building such systems. But each assumes information fits in a **context window** or can be retrieved by a **single search call**. When answers require chaining facts across documents, or when multiple agents need to share and build upon a common world model, the pattern needs an infrastructure layer underneath it. That layer is the knowledge graph.

A. The Problem in Concrete Terms

Consider a **multi-agent system** performing competitive intelligence. An orchestrator delegates to five worker agents: a **pricing analyst**, a **product analyst**, a **financial analyst**, a **marketing analyst**, and a **strategic synthesizer**. Each worker processes a different slice of documents and produces findings. The **strategic synthesizer must chain facts across all five analyses** — "the competitor whose **pricing dropped 15%** is the same one whose patent filing suggests a new product line, and whose quarterly filing shows R&D spending doubled." No single worker saw all three facts. If the workers communicate only through the orchestrator's context window, that window grows linearly with the number of workers and eventually exceeds what even the largest model can hold. But if the workers write their findings as entities and relations into a shared knowledge graph, the synthesizer can traverse the graph to discover the connection without any of the intermediate context.

B. Contributions

This note makes three contributions. First, we present the complete pipeline (Fig. 1) built entirely from Claude API calls — with no

trained models, no external NLP libraries, and no graph database required. Second, we map each stage onto Anthropic's documented agent patterns, showing where a knowledge graph serves as shared memory, grounding layer, or persistent world model. Third, we report precision/recall against a gold set, discuss model selection, and give scaling guidance for production graphs.

II. BACKGROUND

A. Classical Knowledge Graph Construction

The traditional pipeline has three stages, each requiring its own trained model and labeled data. Named Entity Recognition (NER) tags spans of text with labels (PERSON, ORG, LOC). Relation Extraction classifies pairs of tagged spans into relation types (works_at, located_in). Entity Resolution merges mentions of the same real-world entity across documents, typically using string similarity, blocking rules, and hand-tuned thresholds. Each stage is expensive to build, expensive to maintain as data shifts, and brittle when moved to a new domain. The knowledge graph community has spent two decades on this pipeline; Claude collapses it into three prompts.

The brittleness deserves emphasis. A NER model trained on news articles fails on legal contracts; a relation classifier tuned for biomedical literature produces nonsense on financial filings; entity-resolution heuristics calibrated for English names break on transliterated names from other scripts. Every domain shift requires new labeled data, new training, and new evaluation. The promise of the LLM-based pipeline is that the same model, the same schema, and the same prompt work on any domain Claude can read — which is nearly all of them. The adaptation cost drops from weeks of labeling and training to hours of prompt tuning.

B. Anthropic's Agent Patterns

Anthropic's engineering team documents five composable patterns for building agentic systems. The *augmented LLM* is a model enhanced with retrieval, tools, and memory — the atom every agent is built from. *Prompt chaining* decomposes a task into fixed steps with programmatic gates between them. *Routing* classifies an input and sends it to a specialized follow-up. *Orchestrator-workers* has a central LLM dynamically breaking down a task, delegating to workers, and synthesizing results. *Evaluator-optimizer* runs one LLM that generates while another evaluates and feeds back in a loop. Internal research shows multi-agent systems outperform single agents by 90.2% on tasks requiring multiple independent directions — but they consume 10–15× more tokens and require careful context management. The knowledge graph is a structural answer to that management problem.

C. Where the Graph Fits

A knowledge graph serves multi-agent systems in three distinct roles. As *shared memory*: when an orchestrator delegates to workers, each operating in its own context window, the graph is the common ground they all read from and write to — replacing the fragile alternative of passing summaries through the orchestrator's bottleneck. As *grounding layer*: when an evaluator-optimizer loop runs, the evaluator checks the generator's claims against graph edges with explicit provenance, grounding judgment in extracted facts rather than model estimation. As *persistent world model*: when a loop runs overnight, the graph survives context-window flushes the way a state file survives a process restart — the agent forgets, the graph does not.

D. RAG vs. Knowledge Graph

RAG retrieves chunks of text by semantic similarity to a query, then feeds them into the context window. This works for single-hop questions where the answer exists in one passage. It fails for multi-hop questions where the answer requires chaining facts from passages that share no lexical or semantic similarity with the query — or with each other. A knowledge graph bridges this gap: the entity that connects two otherwise unrelated documents is an explicit node with edges to both, and graph traversal discovers the connection regardless of surface-form similarity. The two approaches are complementary: RAG is cheap and effective for direct retrieval, while the knowledge graph handles the structural reasoning RAG cannot.

III. ENTITY AND RELATION EXTRACTION

Classical NER tags spans; classical relation extraction classifies span pairs. We collapse both stages into a single Claude call per document. The key is *structured outputs*: we define the output shape as a Pydantic model and pass it to `client.messages.parse()`. Claude's response is guaranteed to validate against that schema and comes back as a typed Python object — no regex, no JSON decode errors, no defensive checks.

```
EntityType = Literal["PERSON", "ORGANIZATION",
                    "LOCATION", "EVENT", "ARTIFACT"]

class Entity(BaseModel):
    name: str
    type: EntityType
    description: str # one-line, for disambiguation

class Relation(BaseModel):
    source: str
    predicate: str # short verb phrase
    target: str

class ExtractedGraph(BaseModel):
    entities: list[Entity]
    relations: list[Relation]
```

A. The Extraction Prompt

The prompt asks Claude to: (1) extract only entities central to the document — skip incidental mentions; (2) write a one-sentence description grounded in this document — these descriptions are the disambiguation signal for entity resolution; (3) use short verb phrases as predicates ("commanded," "launched from," "part of"); (4) ensure every relation connects two extracted entities. The descriptions are critical: "Armstrong — first person to walk on the Moon" and "Armstrong — jazz trumpeter" have the same name but must not merge. The description replaces what a trained classifier would have learned from domain-specific labeled data.

B. Why Structured Outputs Matter

Without structured outputs, the extraction pipeline must parse free-form text, handle malformed JSON, and validate types at runtime. Each of these is a failure point that scales linearly with corpus size. With structured outputs, the schema is the contract: the API call either returns a valid `ExtractedGraph` object or raises an error. No parsing, no validation, no silent corruption. This is the same principle Anthropic's agent guidance calls "crafting the agent-computer interface" — making tools that are hard to misuse. The difference between a pipeline that processes ten documents reliably and one that processes ten thousand reliably is almost entirely in

the robustness of the interface between stages. Structured outputs make that interface a type-checked contract rather than a hope.

C. The Full Extraction Prompt

The extraction prompt deserves close reading because its wording directly controls the precision/recall tradeoff. The prompt asks Claude to extract *only entities that are central to what this document is about* — this is a precision-favoring instruction that trades recall for noise reduction. On a small corpus where recall matters, weakening this to "extract all mentioned entities" increases recall at the cost of many peripheral mentions that clutter the graph without adding structural value. On a large corpus where precision matters — because every false entity spawns false relations — the "central only" instruction is correct.

```
EXTRACTION_PROMPT = """Extract a knowledge graph from
the document below.
```

```
<document>
{text}
</document>
```

Guidelines:

- Extract only entities that are central to what this document is about – skip incidental mentions.
- For each entity, write a one-sentence description grounded in this document. These descriptions are used later to disambiguate entities with similar names.
- Predicates should be short verb phrases ("commanded", "launched from", "part of").
- Every relation must connect two entities you extracted.

The four guidelines serve four distinct purposes. The first controls recall. The second provides disambiguation context for resolution. The third constrains predicate vocabulary to make the graph traversable (a predicate like "was involved with" is too vague to reason over; "commanded" is not). The fourth is a structural constraint that prevents orphaned edges — a common failure mode where the model produces a relation mentioning an entity it did not extract, creating a dangling reference. Each guideline addresses a specific failure mode observed in earlier iterations; the prompt is the product of exactly the feedback loop the evaluation harness enables.

D. The API Call

The implementation is five lines:

```
def extract(text: str) -> ExtractedGraph:
    response = client.messages.parse(
        model=EXTRACTION_MODEL, # claude-haiku-4-5
        max_tokens=2048,
        messages=[{"role": "user",
                    "content": EXTRACTION_PROMPT
                    .format(text=text)}],
        output_format=ExtractedGraph,
    )
    return response.parsed_output
```

The `output_format` parameter is the entire interface specification. Claude must return an object that validates against `ExtractedGraph`, which contains lists of `Entity` and `Relation` objects, each with typed fields. The result is a Python object with attribute access — `result.entities[0].name` — not a dictionary that might be missing keys. This is the structured-output contract: the "training data" for the extractor is the schema itself.

C. Results on the Apollo Corpus

On a six-document Apollo corpus (Wikipedia summaries of Apollo program, Apollo 11, Neil Armstrong, Saturn V, Buzz Aldrin, and Kennedy Space Center), Haiku extracted 36 raw entities and 34 relations (Table I). The same real-world entity appeared under different surface forms across documents — "Neil Armstrong" and "Neil Alden Armstrong," "Buzz Aldrin" and "Edwin Aldrin" — which is the entity resolution problem the next stage solves.

TABLE I
EXTRACTION RESULTS BY DOCUMENT

Document	Entities	Relations
Apollo program	8	7
Apollo 11	6	5
Neil Armstrong	3	2
Saturn V	5	4
Buzz Aldrin	6	6
Kennedy Space Center	8	10

IV. ENTITY RESOLUTION

Raw extraction gives overlapping mentions: "NASA" and "National Aeronautics and Space Administration," "Neil Armstrong" and "Armstrong," "the Moon" and "Moon." Building a graph directly from this produces a fractured mess where the same concept splits across disconnected nodes.

Traditional approaches use string similarity (edit distance, Jaccard on tokens) plus blocking rules. That works for typos but fails on "Edwin Aldrin" vs. "Buzz Aldrin" — two names with zero character overlap that refer to the same person. We instead ask Claude (Sonnet, for its stronger reasoning) to cluster entities of each type, using the one-line descriptions from extraction as disambiguation context.

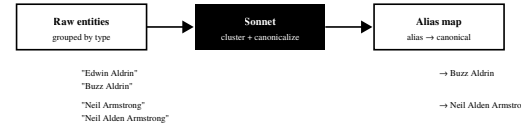


Fig. 2. Entity resolution via Claude. Entities grouped by type, clustered by Sonnet using extraction descriptions as context, mapped to canonical forms.

```
class Cluster(BaseModel):
    canonical: str # most complete form
    aliases: list[str] # all surface forms

class ResolvedClusters(BaseModel):
    clusters: list[Cluster]
```

A. Resolution Results

On the Apollo corpus, resolution compressed 24 unique surface forms to 22 canonical entities — catching "Edwin Aldrin" → "Buzz Aldrin" and "Neil Armstrong" → "Neil Alden Armstrong," cases where string similarity would have failed entirely.

B. Two Failure Modes

Two failure modes are worth monitoring in any resolution stage. First, a raw name left out of every cluster silently vanishes from the graph, because the alias map has no entry for it — a production resolver should fall back to a single-element cluster for unmatched

names so nothing is lost. Second, over-merging can fold a specific entity like "Gemini 12" into the broader "Project Gemini" because the descriptions overlap. The first loses nodes; the second loses precision. Both are worth spot-checking, and the evaluation harness (Section VIII) gives the feedback loop to catch them.

C. Why Descriptions Are the Key

The disambiguation power of the resolution stage comes entirely from the one-line descriptions written during extraction. Without them, the resolver sees only names and must guess from context; with them, it has a semantic signal that is explicit, per-entity, and per-document. This is why the extraction prompt specifies "write a one-sentence description grounded in this document" — the description is not metadata but a first-class input to resolution. An extraction prompt that skips descriptions produces names Claude can cluster only by surface form, falling back to exactly the string-similarity failure mode the whole approach is designed to avoid.

D. The Full Resolution Prompt

The resolution prompt is worth reading in full because its structure determines how well ambiguous cases are handled:

```
RESOLVE_PROMPT = """Below are {entity_type} entities
extracted from several documents. Some are different
surface forms of the same real-world entity.
```

```
<entities>
{entity_list}
</entities>
```

```
Cluster them. Each input name must appear in exactly
one cluster's aliases list. Entities that are
genuinely distinct get their own single-element
cluster. Use the descriptions to avoid merging
entities that merely share a name. The canonical
name should be the most complete, unambiguous
form."""
```

The critical constraints are: (1) every input name must appear somewhere — preventing silent loss; (2) genuinely distinct entities get their own cluster — preventing over-merging; (3) descriptions must be used — preventing surface-form-only matching; (4) the canonical form should be the most complete — ensuring downstream consumers see the most informative name. The prompt processes entities of one type at a time, which keeps the input focused and makes the clustering task tractable even for large entity sets. For PERSON entities in the Apollo corpus, the input is a list of ten names with descriptions; for ORGANIZATION entities, seven names. Each type gets its own Sonnet call.

E. Resolution as a Composable Agent

In a multi-agent architecture, the resolver is itself a specialized agent — it takes input (raw entity lists), applies judgment (which names refer to the same thing), and produces structured output (clusters with canonical forms). It can be run as a worker in an orchestrator-workers pattern, receiving entity batches from the extraction workers and writing canonical mappings to the shared graph. Because the resolution prompt and schema are fixed, the resolver can be deployed as a stateless function that processes batches independently, making it easy to parallelize across entity types. This composability is what makes the pipeline scalable: extraction, resolution, and summarization are three independent agents coordinated by a simple sequential workflow.

V. GRAPH ASSEMBLY AND SUMMARIZATION

With a clean alias map, every relation endpoint is rewritten to its canonical form and loaded into a NetworkX MultiDiGraph. We use a MultiDiGraph because two entities can be connected by several distinct predicates ("launched from" and "operated by"), and direction matters ("Armstrong commanded Apollo 11" is not the same edge as "Apollo 11 commanded Armstrong"). Each node carries its type, source documents, and mention count; each edge carries its predicate and provenance document.

The Apollo graph: 22 nodes, 34 edges, 1 connected component. The single connected component is itself evidence that resolution worked — fragmented islands would indicate variants that should have merged but did not. Hub nodes (Apollo program and Apollo 11, each with degree 9) are the entities that tie the corpus together; node size in a visualization scales with degree, making the hubs visually obvious and giving the builder a quick diagnostic of corpus structure.

A. Entity Summarization

Each node initially carries only the one-line description from whichever document mentioned it first. For hub nodes — those appearing in many documents — we pool every mention, add the graph neighborhood as context, and have Sonnet synthesize a proper profile:

```
class TimeRange(BaseModel):
    start: str # YYYY or "unknown"
    end: str # YYYY or "ongoing"

class EntityProfile(BaseModel):
    summary: str # 2-3 paragraphs
    key_facts: list[str] # 3-5 atomic, traceable facts
    time_range: TimeRange
```

This is the step that turns a graph of labels into a graph of knowledge. The summaries become the node content you surface in search results or feed to downstream agents. Structured time ranges enable temporal reasoning; traceable key facts give the evaluator in an evaluator-optimizer loop something concrete to check against. For the Apollo program node, summarization synthesized information from all six source documents into a three-paragraph profile with five key facts and a time range of 1960–1973 — information that no single document contained in full.

B. When to Summarize

Summarization is expensive — it requires pooling multiple documents and the graph neighborhood into one Sonnet call per entity — so it should be applied selectively. The natural criterion is degree: summarize the top-k nodes by degree, which are the entities that tie the most documents together and benefit most from cross-document synthesis. A practical cutoff is degree ≥ 3 (mentioned in at least two documents from different directions); below that, the single-document description is usually sufficient.

C. The Summarization Prompt

The summarization prompt deserves attention because it is the most complex in the pipeline — it must synthesize across multiple sources while remaining grounded in each:

```
SUMMARIZE_PROMPT = """Generate a knowledge-graph
profile for this entity.

Entity: {name} ({etype})

Source excerpts mentioning this entity:
{excerpts}

Known relations in the graph:
{relations}

Write a 2-3 paragraph factual summary synthesized
from the excerpts, resolving any contradictions by
preferring the most specific claim. Include 3-5
atomic key facts, each traceable to the sources.
For the time range, use YYYY or YYYY-MM format.
Do not invent facts not supported by the
excerpts."""
```

The instruction to "resolve contradictions by preferring the most specific claim" is critical for graph quality. When two documents disagree — one says "Armstrong walked on the Moon in 1969" and another says "Armstrong was part of the first lunar landing" — the summarizer should pick the specific claim rather than the vague one. The instruction to "not invent facts" guards against the summarizer synthesizing plausible but unsupported claims from the general pattern of the data. Both instructions are the summarization analogues of the evaluator's "assume broken until proven otherwise" — defaults to caution.

D. What Summarization Produces

For the Apollo program hub node (degree 9, mentioned in all six documents), the summarizer produced a three-paragraph profile spanning the program's conception in 1960, its dedication to Kennedy's goal in 1961, the first lunar landing in 1969, and the program's use of the Saturn V rocket from Launch Complex 39 at Kennedy Space Center. The five key facts were: conception in 1960, Kennedy's congressional address, first lunar landing in 1969, Saturn V's operational period 1967–1973, and KSC Launch Complex 39 as the launch site. The time range was 1960–1973. No single source document contained all of this; the profile is a genuine synthesis traceable to specific sources — and it is exactly what a downstream agent or search result would surface.

E. Graph Diagnostics

Before querying the graph, a few diagnostic checks are worth running. The number of weakly connected components reveals whether entity resolution left islands — a single component means all entities are reachable from each other, which is the goal. The degree distribution reveals the hub structure: a power-law-like distribution (a few high-degree nodes, many low-degree nodes) is typical of a well-extracted corpus, while a flat distribution suggests either a very homogeneous corpus or an extraction prompt that is treating all mentions equally rather than distinguishing central from peripheral entities. The ratio of edges to nodes gives a density measure: a ratio below 1.0 means the graph is sparse (many isolated entities), while a ratio above 2.0 means the graph is richly connected. The Apollo graph (34 edges / 22 nodes = 1.55) is in the healthy middle range.

VI. MULTI-HOP QUERYING

The payoff of building a knowledge graph is multi-hop reasoning: answering questions that require chaining facts from documents

that share no lexical overlap. "Which locations are connected to people who flew on Apollo 11?" needs person → mission edges from one document and person → location edges from another, then the resolver to have unified the person nodes so those edges actually meet.

The querying mechanism is simple: serialize a relevant subgraph (the k-hop neighborhood of a seed entity) as triples, and let Claude reason over it. The grounded answer differs from the ungrounded one in a way that matters for multi-agent systems: every claim cites a specific edge from a specific document. On a private corpus where Claude has no prior knowledge, only the grounded answer works at all.

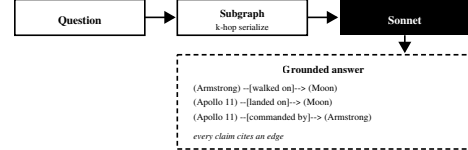


Fig. 3. Graph-grounded querying. A question seeds a subgraph; Sonnet reasons over serialized triples and cites specific edges.

A. Grounded vs. Ungrounded Comparison

The comparison is instructive. Without graph context, Claude draws on pretraining and produces a comprehensive answer about Apollo 11 crew members and their connections to various locations — birthplaces, universities, military bases — pulling from its training data. The answer is plausible and probably correct for a famous topic. With graph context, the answer is constrained to extracted edges: "The only person-location relationship supported by the graph is Neil Armstrong → walked on → the Moon." The grounded answer is less impressive but infinitely more useful: it is traceable, it is limited to what the corpus actually says, and it explicitly flags what the graph does not contain. On a private corpus, only the second kind of answer works at all.

B. Subgraph Selection

The choice of k (the number of hops from the seed entity) controls the tradeoff between coverage and noise. At k=1 the subgraph contains only direct neighbors — fast and focused but misses indirect connections. At k=2 it contains neighbors of neighbors — the sweet spot for most multi-hop questions, capturing the chains that make the graph valuable. At k=3 and beyond the subgraph grows rapidly and may exceed the context window, at which point the serialized triples need filtering or summarization before being fed to Claude. For the Apollo corpus, k=2 from any hub node captures nearly the entire graph (22 nodes, 34 edges), which fits comfortably in a single Sonnet call.

VII. KNOWLEDGE GRAPHS IN MULTI-AGENT ARCHITECTURES

Anthropic documents five agent patterns. Each has a natural integration point with a knowledge graph (Table II).

TABLE II
AGENT PATTERNS AND KNOWLEDGE GRAPH INTEGRATION POINTS

Pattern	KG role	How it helps
Augmented LLM	Retrieval source	Graph traversal replaces vector search for multi-hop questions; the LLM queries the graph as a tool.
Prompt chaining	Gate signal	

Pattern	KG role	How it helps
<i>Routing</i>	Classifier input	Between chain steps, a graph query checks whether new entities conflict with existing nodes.
<i>Orchestrator-workers</i>	Shared memory	Entity type and degree from the graph route a query to the right specialist without an LLM call.
<i>Evaluator-optimizer</i>	Grounding layer	Workers read from and write to the graph; the orchestrator's window stays clean.
		The evaluator checks claims against graph edges with provenance.

A. As Shared Memory for Orchestrator-Workers

When an orchestrator delegates to five worker agents, each operates in its own context window. The classical problem is context management: "context grows too complex for one agent to manage effectively, creating performance bottlenecks as agents struggle to maintain coherence." The knowledge graph solves this structurally. Instead of passing summaries through the orchestrator's window — which grows linearly with the number of workers — each worker reads the subgraph relevant to its task and writes new entities and relations back. The orchestrator's context stays small; the shared state lives in the graph, queryable by any agent at any time.

This is the multi-agent analogue of the "session is not the context window" principle from Anthropic's Managed Agents architecture. The session — here, the knowledge graph — is durable, append-only, and interrogable by positional slice. It does not vanish when a worker's context is flushed. For a team implementing the hierarchical supervisory pattern, the graph replaces the fragile chain of summaries that the supervisor must otherwise maintain, and it gives every specialist agent a shared world model without requiring the supervisor to mediate every cross-domain question.

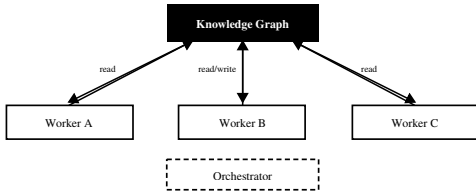


Fig. 4. Knowledge graph as shared memory for orchestrator-workers. Workers read and write the graph directly; the orchestrator's context stays small.

B. As Grounding Layer for Evaluator-Optimizer

The evaluator-optimizer pattern runs "two AI systems in iterative cycles: one generates content while another evaluates and provides feedback, repeating until quality standards are met." The hard part of any evaluator is its basis for judgment: without ground truth, the evaluator judges "does this look right" rather than "is this right." A knowledge graph gives the evaluator something better — it can query the graph for the specific triple the generator claims and check whether that triple exists, with what predicate, from which source document. This shifts the evaluator from a reader to a fact-checker, and the feedback it gives is no longer "this seems off" but "triple (X, works_at, Y) does not exist in the graph; the graph contains (X, left, Y) from document Z." The evaluator-optimizer loop becomes measurably more effective because its verdicts are grounded in extracted facts rather than model estimation.

Consider a concrete example. A generator agent produces a research summary claiming "Armstrong commanded Gemini 12." An evaluator without graph access might let this pass — the claim is plausible, Armstrong was indeed an astronaut, and Gemini 12 was a real mission. An evaluator with graph access queries for (Neil Alden Armstrong) --[commanded]--> (Gemini 12) and finds no such edge. It does find (Buzz Aldrin) --[flew on]--> (Gemini 12) and (Neil Alden Armstrong) --[commanded]--> (Apollo 11). The feedback is precise: "Armstrong did not command Gemini 12; Aldrin flew on Gemini 12. Armstrong commanded Apollo 11." This is fact-checking, not estimation, and it is available because the graph carries provenance — each edge traces to a specific document, which the evaluator can cite.

B.1. The Evaluator-Optimizer Loop with Graph Grounding

The integration has a specific shape. The generator produces content (a summary, a report, a set of claims). Before the evaluator runs, a graph-query stage serializes the relevant subgraph — the k-hop neighborhood of every entity mentioned in the generator's output. The evaluator then receives both the generated content and the graph context, with explicit instructions to check every factual claim against graph edges and to flag claims that have no supporting edge. The feedback includes not just "this is wrong" but the specific graph evidence that contradicts it. The generator receives this feedback and produces a revised version. The cycle repeats until the evaluator passes all claims — or until a claim is flagged as absent from the graph entirely, which is escalated to a human rather than silently accepted or rejected.

C. As Persistent World Model for Loops

A self-improving loop needs memory that survives context-window flushes. The graph is that memory. New documents arriving overnight are extracted, resolved against the existing canonical set (not against each other), and their edges are added. An entity is re-summarized only when its source-document set changes materially. The loop's state file records which documents have been processed and which entities need re-summarization; the graph itself is the world model that accumulates across runs. This is the knowledge-graph equivalent of "the agent forgets, the repo does not."

D. Integration with Collaborative and Hierarchical Systems

The integration extends beyond the five canonical patterns. In a collaborative (peer-to-peer) system where agents communicate directly without a central coordinator, the knowledge graph serves as the shared blackboard — the central knowledge repository that all agents can read from and write to, providing collective memory that persists across interactions. In a hierarchical system where a supervisor delegates to specialists who may have their own sub-agents, the graph can be segmented by domain: each specialist writes to its own subgraph, and the supervisor queries across subgraphs to find cross-domain connections. The graph schema — entity types, relation predicates, provenance tracking — provides the structure that makes this segmentation clean and the cross-domain queries possible.

VIII. EVALUATION

Knowledge graph quality is measured with precision and recall against a gold set. The evaluation scores two things: raw extractor output, and the same entities after resolution (Table III). When the resolver picks a canonical form the gold set recognizes, resolved

recall climbs. When it picks a verbose form the gold set does not cover — say "Neil Alden Armstrong" — resolved recall can drop, which is a scoring artifact, not a resolver bug. The fix is to extend the alias map whenever you see a canonical form the scorer does not recognize.

TABLE III
EXTRACTION QUALITY AGAINST GOLD SET

Document	Raw F1	Precision	Recall	Resolved R
Apollo 11	0.71	1.00	0.55	0.55
Neil Armstrong	0.55	1.00	0.38	0.38

Precision is perfect (1.00) — everything Haiku extracted was correct. Recall is lower (0.38–0.55), meaning the extractor missed some entities the gold set considers important. The missed entities fall into two categories: entities mentioned only in passing ("Purdue University" in the Armstrong article) that the prompt correctly filtered as non-central, and entities from other documents that appear in the gold set's broader scope ("Saturn V" in the Apollo 11 gold). This is a prompt-tuning problem, not a model limitation: adjusting the "extract only central entities" instruction trades recall for precision, and the evaluation harness gives you the feedback loop to make that trade deliberately.

A. The Evaluation Feedback Loop

The evaluation harness — change the extraction prompt, rerun the scorer, watch the F1 move — is the mechanism that turns a demo into a production system. This loop is the same shape as a self-improving agentic loop: act (extract), observe (score), learn (tune the prompt), repeat. The knowledge graph is both an output of such a loop and the infrastructure that makes it work. A team that ships the pipeline without the evaluation harness has no way to know whether prompt changes improve or degrade quality, and no way to catch the slow drift that occurs as the corpus evolves.

B. What the Numbers Mean in Practice

Perfect precision (1.00) means the extractor is conservative — everything it finds is correct. The cost of this conservatism is lower recall (0.38–0.55), meaning it misses some entities the gold set considers important. In a production system, this is usually the right tradeoff: false positives (wrong entities in the graph) are harder to detect and more damaging than false negatives (missing entities), because a wrong entity spawns wrong relations that propagate through multi-hop reasoning. A missing entity produces an incomplete but correct graph; a wrong entity produces a graph that actively misleads.

The missed entities are instructive. "Purdue University" appears in the Neil Armstrong article but is peripheral to the article's main topic — the extraction prompt correctly filtered it as non-central. "Saturn V" appears in the Apollo 11 gold set but was not mentioned centrally enough in the Apollo 11 summary for Haiku to extract it — it was extracted from the Saturn V article itself. This is a scope mismatch between the per-document extraction and the cross-document gold set, not an extraction failure. Adjusting the prompt to "extract all mentioned entities" would capture Saturn V in the Apollo 11 document but would also capture dozens of peripheral mentions that add noise without structural value. The evaluation harness lets you make this tradeoff explicitly, document it, and revisit it as the corpus changes.

C. Scoring Relations

Entity evaluation is necessary but not sufficient. Relation evaluation is harder because predicate wording varies — "commanded" and "led" and "was commander of" all describe the same relation. The cookbook's evaluation script scores relations on (source, target) pairs, ignoring predicate wording, which gives an upper bound on relation recall. A more sophisticated scorer would define equivalence classes of predicates, but for most production purposes the (source, target) match is sufficient to detect structural errors — missing connections or wrong connections — which are more impactful than predicate-wording differences.

IX. SCALING GUIDANCE

The notebook processed six documents in memory. Production knowledge graphs are built from thousands. Four considerations govern the transition.

A. Extraction Cost

Haiku is cheap enough to run on large corpora, but prompt caching cuts costs further when the extraction schema and instructions stay fixed — cache the system prompt and schema, pay full price only for the document text. The Message Batches API gives 50% off for jobs that can tolerate up to 24 hours of latency. For a corpus of 10,000 documents averaging 2,000 tokens each, the extraction cost at Haiku rates is measured in single-digit dollars — a fraction of what training and running a dedicated NER model would cost.

B. Resolution at Scale

Feeding ten thousand PERSON entities to Claude in one prompt does not work. Block first: group candidates by cheap signals (same last name, overlapping tokens, embedding similarity) so Claude only arbitrates within small blocks. The resolution prompt works unchanged on blocks of 50–100. The blocking itself can be implemented with a simple inverted index on name tokens — no model call required. This hybrid approach — cheap deterministic blocking plus expensive LLM arbitration within blocks — is the same pattern the pipeline uses throughout: keep the model for the parts that require judgment, and use deterministic logic for everything else.

C. Incremental Updates

When a new document arrives, extract its entities, resolve them against the existing canonical set (not against each other), and add only the new edges. Re-summarize an entity only when its source-document set changes materially. This is the graph analogue of the loop's state file: the graph accumulates rather than rebuilds.

D. Storage

NetworkX is fine to a few hundred thousand edges. Beyond that, the schema maps directly onto a property graph (Neo4j, Neptune) or three Postgres tables: `entities(id, name, type, summary)`, `relations(source_id, target_id, predicate)`, `aliases(entity_id, alias)`. The extraction and resolution code does not change — only the persistence layer does. For teams already running Neo4j, the mapping is direct: each entity becomes a labeled node with properties for type, description, and source documents; each relation becomes a typed edge with a provenance property. For teams without a graph database, the Postgres approach is simpler: three tables, standard SQL, and graph queries

implemented as recursive CTEs. The choice between them is an infrastructure decision, not a pipeline decision.

E. Chunking Strategy for Long Documents

The cookbook uses Wikipedia summaries — short texts that fit comfortably in a single extraction call. Production documents are longer: legal contracts, research papers, technical documentation. For these, the document must be chunked before extraction, and the chunks must be designed so that entities and their relations fall within the same chunk. Naive chunking by token count splits entities from their context; semantic chunking by paragraph or section boundary preserves the co-occurrence structure that the extractor depends on. A practical rule: chunk at section boundaries, with overlap of one paragraph, so that an entity mentioned at the end of one section is still in context for relations described at the beginning of the next. The extraction prompt works unchanged on chunks; the additional step is deduplicating entities across chunks of the same document before resolution, which is a lightweight per-document resolution pass using exact string matching.

F. Monitoring in Production

A production knowledge-graph pipeline needs four monitoring signals. First, *extraction rate*: the number of entities and relations extracted per document. A sudden drop suggests the corpus has shifted to a domain the prompt handles poorly; a sudden spike suggests the extractor is over-extracting peripheral mentions. Second, *resolution compression ratio*: the number of raw surface forms divided by canonical entities. A ratio near 1.0 means the corpus uses consistent naming and resolution is doing little; a ratio above 2.0 means the corpus has significant naming variation and resolution is earning its cost. Third, *graph connectivity*: the number of connected components and the size of the largest. A growing number of disconnected components suggests resolution is missing cross-document links. Fourth, *query latency*: the time from question to grounded answer. For real-time applications, subgraph serialization should be pre-computed for high-traffic seed entities; for batch applications, latency is less critical but cost per query matters more.

TABLE IV
MODEL SELECTION FOR PIPELINE STAGES

Stage	Model	Rationale
Extraction	Haiku	High volume, schema-constrained; speed and cost dominate.
Resolution	Sonnet	Weighing conflicting evidence; reasoning quality dominates.
Summarization	Sonnet	Synthesizing across documents; nuance matters.
Querying	Sonnet	Multi-hop reasoning over serialized triples.

X. RELATED WORK

Knowledge graph construction from text has a long history in the NLP community, from early information extraction systems through the statistical NER models of the 2000s to the neural approaches of the 2010s. The distinguishing feature of the LLM-based approach presented here is the elimination of domain-specific training: the same model, schema, and prompt work across domains, with adaptation achieved through prompt tuning rather

than retraining. This trades fine-grained control over entity types (which a trained model can learn from labeled examples) for generality and ease of deployment.

The use of LLMs for entity resolution specifically has been explored by several groups, with the common finding that LLMs handle the "hard" cases (nicknames, abbreviations, cross-lingual variants) that string-similarity methods miss, while being slower and more expensive for the "easy" cases (typos, capitalization variants) that edit distance handles cheaply. The hybrid approach used here — LLM resolution within blocks formed by cheap deterministic signals — is a practical synthesis of these findings.

The integration of knowledge graphs with multi-agent systems is less explored in the literature. The closest precedent is the use of shared knowledge bases in multi-agent reinforcement learning, where agents maintain a common state representation. More recently, the concept of a "blackboard architecture" — where agents communicate through a shared knowledge repository — has been applied to LLM-based multi-agent systems. Anthropic's Building Effective AI Agents documentation describes this as one variant of collaborative systems, where a "blackboard architecture provides shared knowledge repositories where all agents can read from and write to a central knowledge repository acting as collective memory." The contribution of this note is to show that a knowledge graph built from prompts provides a natural, structured, provenance-carrying implementation of that blackboard.

A. Relationship to RAG

Retrieval-augmented generation (RAG) and knowledge graphs are complementary, not competing, approaches to grounding LLM outputs. RAG excels at finding relevant passages by semantic similarity — given a question, it retrieves the chunks most likely to contain the answer and feeds them into the context window. This works well for single-hop questions where the answer exists in one passage. It breaks down for multi-hop questions where the answer requires chaining facts from passages that are semantically dissimilar to each other and to the question. The knowledge graph bridges this gap: the entity that connects two otherwise unrelated documents is an explicit node with edges to both, and graph traversal discovers the connection regardless of surface-form similarity. In practice, the two approaches are best used together: RAG for direct retrieval, the knowledge graph for structural reasoning, and the LLM for synthesizing across both sources.

B. Relationship to Closed-Loop Optimization

The evaluation feedback loop of this pipeline (change prompt → rerun scorer → watch F1 move) is structurally identical to the closed-loop optimization pattern studied in the compiler and systems literature, where an LLM proposes transformations, the environment measures their effect, and the LLM refines its next proposal from that feedback. The "compiler" in our case is the gold-set scorer; the "transformation" is the extraction prompt; the "measured effect" is the F1 score. The same principle — that a loop's intelligence lives in the quality of the environmental feedback, not in the model — applies here: a pipeline with a good scorer improves itself; a pipeline without one drifts.

C. Structured Outputs as the Enabling Capability

The entire pipeline depends on a single API capability: structured outputs that guarantee the model's response validates against a Pydantic schema. Without this, every extraction call returns free-form text that must be parsed, validated, and coerced into the

correct types — a process that fails silently at scale. Structured outputs eliminate this failure class entirely, making it feasible to run the pipeline on thousands of documents without a single parse error. This is the capability that makes the prompt-as-training-data approach practical rather than merely theoretical, and it is why the pipeline described here would not have been practical two years ago.

XI. DISCUSSION

A. What the Graph Replaces

The complete pipeline replaces four distinct trained systems, each requiring labeled data per domain, each breaking when moved to a new corpus. The Pydantic schema is the only "training" required. This is the same simplification pattern Anthropic's agent guidance recommends: prefer the simplest solution, add complexity only when it demonstrably improves outcomes. A knowledge graph built from prompts is simpler to build, simpler to maintain, and simpler to adapt than one built from trained models.

B. What the Graph Does Not Replace

Judgment. The graph is a structured fact store; it does not decide which facts matter, which entities to trust, or which actions to take. In the context of multi-agent systems, the graph is the shared memory, not the decision-maker. The orchestrator still needs to decide which subgraph to query; the evaluator still needs to decide whether a missing triple is evidence of an error or evidence of a gap in the corpus. The graph moves the basis for these decisions from model estimation to extracted facts, but the decisions themselves remain with the agents — and ultimately with the human who designed them.

C. Limitations

Three limitations are worth stating plainly. First, extraction quality depends on prompt engineering; the evaluation harness gives a feedback loop, but the loop must be run. Second, resolution at scale requires blocking heuristics that are themselves domain-dependent — the clustering prompt is universal but the blocking is not. Third, the graph is only as good as the documents it was built from; a biased or incomplete corpus produces a biased or incomplete graph, and no amount of multi-hop reasoning over a biased graph produces unbiased answers. The graph amplifies the quality of the corpus, just as a loop amplifies the judgment of its builder.

D. Cost Considerations

The cost structure of the pipeline has three regimes. Extraction is cheap per document (Haiku) and scales linearly with corpus size — this is where prompt caching and batching have the most impact. Resolution is moderate — Sonnet calls, but only one per entity type per resolution pass, not one per document. Summarization is expensive per entity (Sonnet with large context) but is applied only to high-degree nodes, so the total cost is sublinear in corpus size. Querying is per-question and per-subgraph, with cost proportional to subgraph size. The dominant cost for a large corpus is extraction; the dominant cost for a heavily-queried graph is querying. Optimizing both independently — Haiku for extraction, Sonnet for querying — is why the two-model approach matters.

E. Operational Discipline

A knowledge graph that runs as part of an unattended loop inherits the same operational concerns as any autonomous system. Three disciplines are worth stating as standing practice. First, *sample the*

graph regularly: pick a random node each day, read its profile, check its edges against the source documents, and verify the provenance chain. This is the graph analogue of reading a sample of the loop's PRs to guard against comprehension rot — the moment you cannot explain why a node has a particular edge, your understanding of the graph has fallen behind its contents. Second, *cap extraction volume before you ship*: a per-run limit on documents processed and entities extracted ensures that a corpus ingestion error (a batch of duplicate documents, a misrouted data feed) cannot produce a run of unbounded cost. Third, *version the schema*: when you change entity types, add predicates, or adjust the extraction prompt, the graph built under the old schema and the graph built under the new one may not be compatible. A production pipeline versions the schema alongside the graph, so that entities extracted under different prompts can be distinguished, compared, and — if necessary — re-extracted.

F. Future Directions

Three extensions are natural. First, *temporal graphs*: adding timestamps to edges so that the graph captures not just what is true but when it was true, enabling questions like "who held this role in Q3 2024?" that the current atemporal graph cannot answer. The EntityProfile already includes a TimeRange; extending relations with a similar field is straightforward in the schema and would let Claude filter subgraphs by time window before reasoning. Second, *confidence scoring*: attaching an extraction-confidence signal to each edge, derived from the model's own uncertainty or from cross-document corroboration (an edge extracted from three independent documents is more trustworthy than one extracted from a single source). This would let the evaluator weight its fact-checks rather than treating all edges as equally reliable. Third, *graph-of-graphs*: in a multi-team setting, each team maintains its own domain graph; a meta-graph records the connections between domain graphs, enabling cross-team reasoning without merging incompatible schemas. This is the knowledge-graph analogue of the federated or peer-to-peer multi-agent architecture, and it inherits the same coordination challenges.

XII. CONCLUSION

We have presented a complete knowledge-graph pipeline built entirely from Claude API calls: extraction via structured outputs replaced a trained NER model plus a trained relation classifier; resolution via Claude clustering caught cases that string similarity misses entirely; summarization gave hub nodes rich profiles synthesized across every document that mentioned them; and querying via serialized subgraphs let Claude answer multi-hop questions with edge-level citations.

We have shown where the knowledge graph slots into each of Anthropic's five documented agent patterns — as retrieval source, gate signal, classifier input, shared memory, and grounding layer — and argued that for multi-agent systems, the graph solves the fundamental problem that each agent's memory dies with its context window. The graph is the durable, queryable, provenance-carrying world model that lets agents share state without passing it through an orchestrator's bottleneck, and that lets evaluators fact-check against extracted edges rather than against their own estimation.

The evaluation harness — change the extraction prompt, rerun the scorer, watch the F1 move — is what turns a demo into a production system. That loop is the same shape as the self-

improving agentic loop: act, observe, learn, repeat. The knowledge graph is both an output of such a loop and the infrastructure that makes it work. The Pydantic schema is the only training data required; the graph works on any domain Claude can read; and the entire pipeline runs with no trained models, no external NLP libraries, and no graph database — just prompts, a schema, and a graph library.

ACKNOWLEDGMENT & SOURCES

This document is an independent synthesis for study, not affiliated with or endorsed by Anthropic. The knowledge-graph pipeline, code, and evaluation methodology are from Anthropic's public cookbook, "**Knowledge Graph Construction with Claude**," available in the claude-cookbooks repository. The agent patterns, decision frameworks, and multi-agent architecture guidance are from "**Building Effective AI Agents: Architecture Patterns and Implementation Frameworks**," Anthropic, 2026. The augmented LLM, prompt chaining, routing, orchestrator-workers, and evaluator-optimizer patterns are from **E. Schluntz & B. Zhang**, "Building Effective Agents," Anthropic Engineering, Dec. 2024. The Managed Agents architecture (brain/hands/session decoupling) is from **L. Martin, G. Cemaj & M. Cohen**, "Scaling Managed Agents," Anthropic Engineering, Apr. 2026. The competitive intelligence example in Section I is illustrative; the Apollo corpus results are from the cookbook. All diagrams are original. Code examples are adapted from the public cookbook for pedagogical clarity. Product capabilities and figures cited reflect source material at time of writing and may have since changed.

APPENDIX: GLOSSARY

TABLE V
TERMS USED IN THIS NOTE

Term	Meaning
<i>Knowledge graph</i>	A structured representation of entities (nodes) and their typed relationships (edges), with provenance.
<i>Structured outputs</i>	API feature that constrains Claude's response to validate against a Pydantic schema, returning typed objects.
<i>Entity resolution</i>	Merging different surface forms of the same real-world entity into a single canonical node.
<i>Multi-hop reasoning</i>	Answering a question that requires chaining facts across multiple edges in the graph.
<i>Subgraph serialization</i>	Converting a portion of the graph into a text format (triples) that can be fed into a context window.
<i>Ground truth</i>	A verdict from the environment (a graph edge with provenance) rather than from model estimation.
<i>Blocking</i>	Grouping entity candidates by cheap signals before expensive LLM resolution within blocks.
<i>Hub node</i>	An entity with high degree (many connections), typically the most important entity in the graph.
<i>Alias map</i>	A dictionary mapping every known surface form of an entity to its canonical name.
<i>Provenance</i>	The source document and extraction context from which a triple was derived.

APPENDIX B

Worked Example: Competitive Intelligence System

To make the multi-agent integration concrete, this appendix walks through a competitive intelligence system that uses a knowledge graph as shared memory across five specialist agents.

System Architecture

The system has one orchestrator and five workers: a pricing agent, a product agent, a financial agent, a marketing agent, and a strategic synthesizer. The orchestrator receives a request ("analyze competitor X's market position") and delegates sub-tasks. Each worker processes a different corpus slice — the pricing agent reads pricing pages, the product agent reads product announcements and patents, the financial agent reads quarterly filings, and so on.

Step 1: Parallel Extraction

Each worker runs the extraction pipeline on its document slice, producing entities and relations specific to its domain. The pricing agent extracts entities like (ProductA, PRODUCT), (PriceTier, PRICING), with relations like (ProductA) --[priced_at]--> (\$99/mo). The product agent extracts (PatentFiling, DOCUMENT), (NewFeatureX, FEATURE), with relations like (CompetitorX) --[filed]--> (PatentFiling). The key: each worker uses the same schema and extraction prompt, with only the entity types extended to include domain-specific types (PRICING, FEATURE, FILING). The schema is the contract that ensures all workers' outputs are compatible.

Step 2: Cross-Worker Resolution

The resolver agent runs after all workers complete, taking the combined entity lists from all five workers and resolving them into canonical forms. The critical connections happen here: the financial agent extracted "Acme Corp" while the product agent extracted "ACME Corporation" and the pricing agent extracted "acme" — all three must resolve to the same canonical node. Without this step, the strategic synthesizer could not discover that the company whose pricing dropped is the same one whose patent filing suggests a pivot, because they would be three disconnected nodes.

Step 3: Graph Assembly

The resolved entities and relations from all five workers are assembled into a single graph. Each edge carries provenance — which worker extracted it, from which document. The graph now contains cross-domain connections that no single worker could have produced: (Acme Corp) has outgoing edges to pricing data, patent filings, quarterly results, marketing campaigns, and strategic indicators, all traceable to their source documents.

Step 4: Strategic Synthesis via Graph Traversal

The strategic synthesizer does not re-read the source documents. Instead, it queries the graph: "show me all entities connected to Acme Corp within 2 hops, with predicates and provenance." The serialized subgraph arrives as triples, and the synthesizer reasons over them to produce its analysis. Every claim in the analysis cites specific graph edges: "Acme Corp reduced pricing by 15% (source: pricing-agent, document: pricing-page-q3.html) while simultaneously filing patent US-2024-XXXX for a new product category (source: product-agent, document: patent-filing.pdf), suggesting a strategy of undercutting incumbents before launching a differentiated offering." The orchestrator's context window never held all five workers' raw outputs — only the synthesizer's graph query results.

Why This Works

The graph is doing three things that neither RAG nor context-passing could do. First, it is *connecting* — the entity resolution

links "Acme Corp" across workers that never communicated directly. Second, it is *compressing* — the synthesizer reads a sub-graph of maybe 50 triples, not five workers' worth of raw documents. Third, it is *grounding* — every triple has provenance, so the analysis is traceable to sources, not to the synthesizer's imagination. The graph is the infrastructure that makes the orchestrator-workers pattern work at scale for cross-document reasoning.

APPENDIX C

Decision Framework: When to Use a Knowledge Graph

Not every multi-agent system needs a knowledge graph. The decision framework below helps practitioners match the infrastructure to the problem.

TABLE VI
WHEN TO USE A KNOWLEDGE GRAPH VS. ALTERNATIVES

Scenario	Right tool	Why
Single-document QA	RAG or direct context	No multi-hop needed; the answer is in one chunk.
Multi-doc, single-hop	RAG with reranking	The answer spans docs but doesn't require chaining.
Multi-doc, multi-hop	<i>Knowledge graph</i>	Chaining facts across docs requires entity-level linking.
Multi-agent, shared state	<i>Knowledge graph</i>	Workers need a shared world model outside context windows.
Evaluator needs ground truth	<i>Knowledge graph</i>	Fact-checking requires structured, provenance-carrying facts.
Overnight loop, persistent memory	<i>Knowledge graph</i>	Memory must survive context flushes across sessions.
Simple classification or routing	Single agent	No cross-document reasoning needed.

The rule of thumb: if your agents need to *chain* facts across sources, *share* structured state, or *ground* their judgments in traceable evidence, the knowledge graph is the right infrastructure. If they need to *retrieve* relevant passages or *classify* inputs, simpler tools suffice. The graph earns its complexity when the alternative — passing raw documents or summaries through context windows — either exceeds the window or loses the connections.

APPENDIX D

Production Readiness Checklist

Before deploying a knowledge-graph pipeline in production, the following checklist ensures the pipeline is robust, observable, and maintainable. Each item maps to a failure mode discussed in the main text.

TABLE VII
PRODUCTION READINESS CHECKLIST

Element	Ask yourself	Failure if missing
<i>Gold set</i>	Do you have a hand-labeled evaluation set for at least two representative documents?	No feedback loop; prompt changes are blind.
<i>Alias map</i>	Does the scorer's alias map cover every canonical form the resolver produces?	Scoring artifacts — recall appears worse than it is.
<i>Schema version</i>	Is the extraction schema versioned alongside the graph?	Incompatible entities from different prompt versions.

Element	Ask yourself	Failure if missing
<i>Extraction cap</i>	Is there a per-run limit on documents processed?	Unbounded cost from corpus ingestion errors.
<i>Resolution fallback</i>	Do unmatched names get single-element clusters?	Silent entity loss — nodes disappear from the graph.
<i>Provenance tracking</i>	Does every edge carry its source document and extraction timestamp?	Ungrounded answers; evaluator cannot fact-check.
<i>Incremental update</i>	Can new documents be added without rebuilding the entire graph?	Rebuild cost scales with corpus size, not delta size.
<i>Connectivity monitor</i>	Do you check connected components after each resolution pass?	Fragmented graph — missed cross-document links.
<i>Summarization trigger</i>	Is re-summarization triggered only when source-document set changes?	Wasted Sonnet calls on unchanged entities.
<i>Human sample</i>	Does someone read a random node profile each day?	Comprehension rot — the graph outgrows understanding.

The first two items (gold set, alias map) give you the evaluation feedback loop. The next three (schema version, extraction cap, resolution fallback) prevent silent data corruption. The next three (provenance, incremental update, connectivity monitor) keep the graph structurally sound. The last two (summarization trigger, human sample) keep costs manageable and understanding current. A pipeline with all ten is production-ready; a pipeline missing any of them has a specific, nameable risk that will surface eventually.

The closing observation is the same one that applies to every autonomous system: the pipeline is not done when it runs; it is done when you can tell, on any given morning, whether what it produced overnight was actually right. The evaluation harness, the provenance tracking, and the human sample are what make that possible. Everything else is infrastructure; those three are judgment.

APPENDIX E

Complete Query Implementation

For reference, the complete query implementation — subgraph serialization, prompt construction, and comparison between grounded and ungrounded answers — is reproduced below. The `serialize_subgraph` function performs a breadth-first traversal from a seed entity, collecting all nodes within k hops, then formats the edges of the induced subgraph as triples. The `ask` function either sends the question directly (ungrounded) or wraps it with graph context (grounded).

```

def serialize_subgraph(center: str, hops: int = 2
                      ) -> str:
    nodes = {center}
    frontier = {center}
    for _ in range(hops):
        nxt = set()
        for n in frontier:
            nxt |= set(G.successors(n))
            nxt |= set(G.predecessors(n))
        frontier = nxt - nodes
        nodes |= frontier
    sub = G.subgraph(nodes)
    lines = [
        f"({s}) --[{d['predicate']}]--> ({t})"
        for s, t, d in sub.edges(data=True)
    ]
    return "\n".join(sorted(set(lines)))

def ask(question: str,
        graph_context: str | None = None) -> str:
    if graph_context is not None:
        prompt = f"""Answer using only the knowledge
graph below. Cite the specific edges that support
your answer.

<graph>
{graph_context}
</graph>

Question: {question}"""
    else:
        prompt = question

    response = client.messages.create(
        model=SYNTHESIS_MODEL,
        max_tokens=500,
        messages=[{"role": "user",
                    "content": prompt}],
    )
    return next(
        b.text for b in response.content
        if b.type == "text"
    )

```

The graph-context prompt is deliberately restrictive: "Answer using *only* the knowledge graph below." This constraint is what makes the answer traceable — the model cannot draw on pretraining and must instead cite specific edges. The "cite the specific edges" instruction produces answers like "(Armstrong) --[walked on]--> (Moon)" rather than prose claims, making it trivial for a downstream evaluator agent to verify each citation against the graph programmatically. This is the mechanistic basis for the grounding-layer integration described in Section VII.B.

For production use, the query function should also return the subgraph itself (as a list of triples) alongside the answer, so that the calling agent or human can see exactly which edges were available. This transparency is what distinguishes graph-grounded answers from RAG-grounded answers: with RAG, the context is a blob of retrieved text that may or may not contain the supporting evidence; with a graph, the context is a set of explicit triples each carrying provenance, and the answer's citations can be verified by simple string matching against the input.